# Problem Statement

## Purpose

The purpose of this case study is to execute and demonstrate the below deployment strategy for a Python full-stack application on AWS:

Manual deployment of the Python full-stack application, which includes setting up the frontend on Amazon S3, deploying backend FastAPI microservices on EC2, configuring a PostgreSQL database on RDS, integrating AWS Lambda for event-driven processing, and enabling monitoring with CloudWatch.

## Objective

The objective is to perform manual deployment for Python full-stack projects on AWS. This will highlight the key implementation steps and AWS services used and assess the effectiveness, efficiency, scalability, and maintainability of this approach. The aim is to guide best practices for cloud-native application deployment on AWS.

## Tools and Technologies

| Tools / Technologies | Description |
|---|---|
| FastAPI | Framework for building microservices |
| Uvicorn | ASGI web server implementation for Python |
| PostgreSQL | Relational database for storing service data |
| Angular/React | A JavaScript framework or library used for developing frontend single-page applications (SPAs) |
| Apache | HTTP Server on EC2 to serve as Proxy Server to Uvicorn |
| Amazon EC2 | Hosts backend microservices and supporting services |
| Amazon S3 | Hosts static frontend assets (Angular/React) |
| Amazon RDS | Managed PostgreSQL database service |
| AWS Lambda | Runs serverless functions |
| AWS CloudWatch | Provides monitoring, logging, and alerts for AWS resources |
| Swagger UI | API documentation, visualization, and interaction tool |

# Functional Requirements

Let's discuss the details of milestone, including the corresponding architecture and completion tasks for AWS-based deployment of Python full-stack projects.

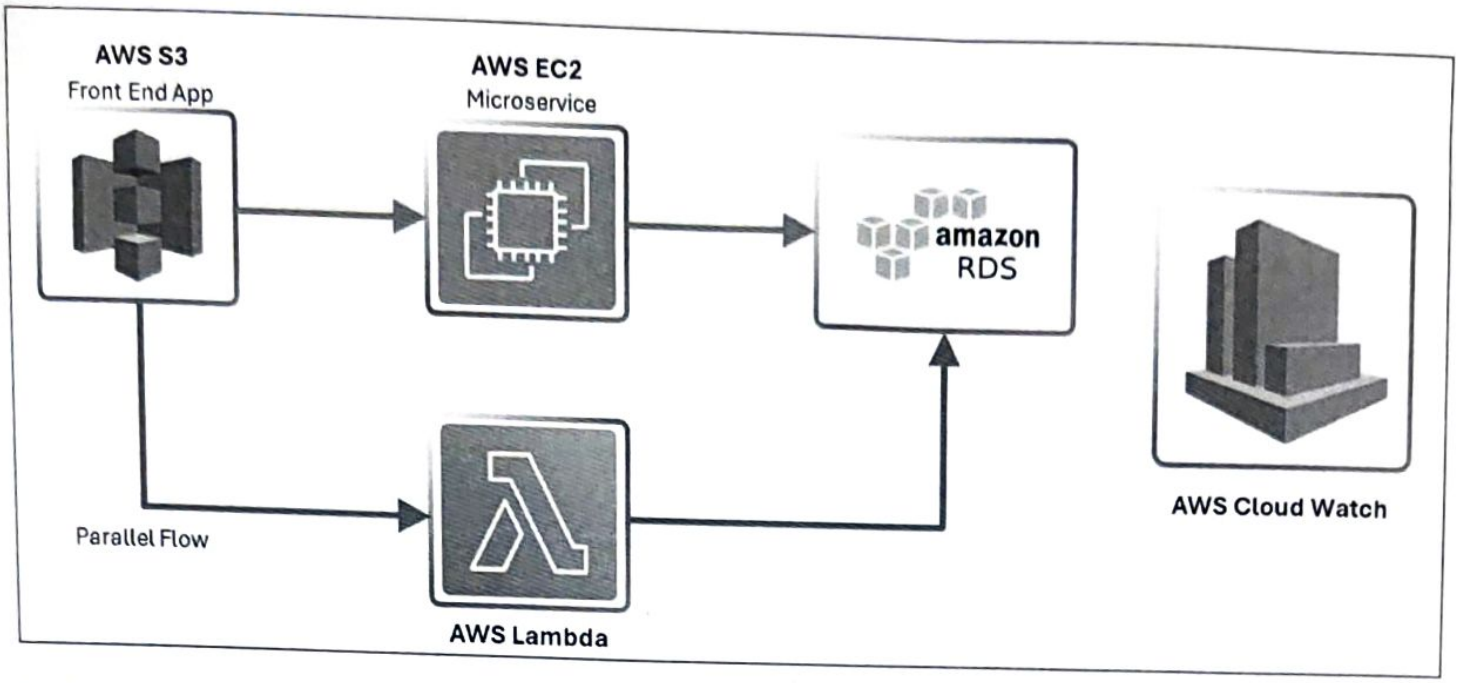
## Manual Deployment of Python Full Stack Project using EC2, S3 bucket, RDS, and Lambda

**Overview**

This milestone focuses on the manual deployment of a Python full-stack application using AWS services (S3, EC2, RDS). The deployment includes setting up the frontend, backend microservices, database, and monitoring, as well as integrating AWS Lambda for event-driven processing.

Copyright © 2026 Deloitte Development LLC. All rights reserved.

## Architecture




## Tasks

- **Set Up Frontend on Amazon S3**
  - Build the frontend artifacts.
  - Upload static frontend assets (HTML, CSS, JS) to an S3 bucket.
  - Configure S3 bucket permissions for public access (if required).
  - Enable static website hosting on S3.

- **Deploy Backend Microservices on EC2**
  - Provision EC2 instances for microservice(s) [AMI-Ubuntu].
  - Install Python, FastAPI Framework, Uvicorn, Apache Server on EC2 instance.
  - Deploy the FastAPI project files to the respective instance.
  - Configure security groups to allow necessary inbound/outbound traffic.
  - Configure Apache Server to work in Proxy and Reverse Proxy mode.
  - Update Apache configuration to route traffic to FastAPI listening port.

- **Set Up Amazon RDS (PostgreSQL)**
  - Launch an Amazon RDS instance with PostgreSQL.
  - Configure database schema and user permissions.
  - Update backend microservice configurations to connect to the RDS instance.

- **Integrate AWS Lambda for Event-Driven Tasks**
  - Create Lambda functions for specific event-driven processes (e.g., processing S3 uploads).
  - Set up S3 event triggers to invoke Lambda functions as needed.
  - Grant necessary permissions for Lambda to access S3.

- **Configure AWS CloudWatch for Monitoring**
  - Set up CloudWatch to collect logs and metrics from EC2 instances and Lambda functions.
  - Create dashboards and alarms for proactive monitoring (Optional).

- **Testing and Validation**
  - Test end-to-end functionality from the user browser through to the backend and database.

Copyright © 2026 Deloitte Development LLC. All rights reserved

- Validate Lambda event triggers and processes.
- Monitor logs and metrics in CloudWatch for any issues.

## Non-Functional Requirements

- The application should have low latency and high throughput.
- The application and data should be secured by Authentication.
- System should be scalable and maintainable.

## Guidelines

- Make use of best coding practices.
- You can make assumptions and incorporate additional features or functionalities.
- A simple but working prototype is preferred over a non-working but good-looking application.
- Add relevant comments for documenting deployment steps, configurations, and architecture for easy reference and onboarding.
- There would be a heavy focus on code quality and robustness of code in general.
- Use separate development, testing, and production environments to avoid conflicts and ensure stability.
- Assign least privileged IAM roles and security groups to all AWS resources.
- Implement regular backups for databases and critical data.
- Leverage only Deloitte GitHub enterprise wherever it is needed.

## Best Practices

| | |
|---|---|
| **Clear and Consistent Resource Naming** | Use clear and consistent naming conventions for AWS resources, services, and code repositories. |
| **Monitoring and Logging** | Leverage AWS CloudWatch for real-time monitoring, centralized logging, and alerting. |
| **Scalability** | Design microservices and infrastructure to support horizontal scaling (EC2, ECS, Lambda). |
| **Testing** | Use tools like Swagger UI, ReDoc and Pytest for thorough API testing and validation before production releases. |
| **Documentation** | Integrate Swagger for up-to-date, interactive API documentation. |

## Milestones

| Milestone# | Milestone Description |
|---|---|
| **Milestone 1** | Set up Frontend app on Amazon S3 |
| **Milestone 2** | Deploy Backend Microservices on EC2 |
| **Milestone 3** | Set Up Amazon RDS (PostgreSQL) |

| | |
|---|---|
| **Milestone 4** | Integrate AWS Lambda for Event-Driven Tasks |
| **Milestone 5** | Configure AWS CloudWatch for Monitoring |
| **Milestone 6** | Test and Validate End-to-End functionality |
| **Technical Interview** | Discussion of Assumptions, Functionalities, and Validations |